

Global Sensitivity & Uncertainty Analysis (GSUA)

Morris (Elementary Effects) for d50 (DP)

Student: Angel Morenu

Instructor: Dr. Rafael Muñoz-Carpena

Date: 2025-11-21

This report is for GSUA workflow used in class ("Bungee → d50") with a simple empirical model for median sediment particle size (**d50**, denoted **DP**).

It includes: data inspection, distribution assignment with histograms, a Morris screening design, model evaluation, and sensitivity measures (μ , μ^* , σ), followed by interpretations and answers to the assignment questions.

Introduction:

The goal of this analysis was to screen which input factors most influence the modelled median particle diameter (d50) using the Elementary Effects (Morris) method. The canonical d50 model in this repository was used; because MATLAB was not available, an equivalent Morris screening was executed in Python and MATLAB-format helper files were produced for portability.

Methods (summary):

- Model: canonical d50 implementation from the notebook; outputs are reported in micrometers (μm).
- Dataset: `d50_for_reg_20211102b.csv` (factor bounds derived from observed data).
- Factors: `ITILLAGE`, `Ein`, `Rmmh`, `Qpin`, `SLENGTH`, `SILTFRAC`.
- Morris settings: levels $p = 6$, $\Delta = p/(2 \cdot (p-1))$, trajectories $r = 20$, RNG seed fixed for reproducibility. `ITILLAGE` was treated as a binary factor (sampled on $[0,1]$ then rounded).

d50 (DP) equation

The model computes PART1 and then scales to DP:

[

$$\text{PART1} = 5.388 + 35.275 \cdot \text{ITILLAGE} + 0.322\sqrt{\text{Ein}} + 0.066 \text{Rmmh} + 0.115 \text{Qpin} \\ - \frac{29.576}{\text{Slength}} - \frac{1.519}{\sqrt{\text{SiltFrac}}}$$

]

Constraints:

- If $PART1 < 1.1$, then set $PART1 = 1.1$.

Final scaling:

- $DP \text{ (cm)} = PART1 / 10000$ (because $1 \text{ cm} = 10,000 \text{ } \mu\text{m}$).
- If you want DP in μm , $DP(\mu\text{m}) = PART1$.

Variable definitions and notes:

- ITILLAGE: 0 = conventional, 1 = no-tillage (binary).
- E_{in} : outgoing sediment load (g/m^2). Enter non-negative values.
- Rmmh: mean rainfall intensity (mm/h).
- Qpin: peak runoff rate (mm/h).
- Slength: slope length (m). Must be > 0 (avoid division by zero).
- SiltFrac: silt fraction (fraction 0–1 or % 0–100). Verify your data: if given as percent, convert ($\text{SiltFrac} = \text{percent}/100$). The formula uses $\text{sqrt}(\text{SiltFrac})$ in the denominator, so SiltFrac must be > 0 .

Implementation tips:

- Enforce small positive lower bounds for Slength and SiltFrac to avoid division by zero (e.g., $\max(\text{value}, 1\text{e-}12)$).
- Treat ITILLAGE as integer $\{0,1\}$ when sampling from continuous ranges.

1. Interpretation of Step A results

Below is a concise table summarizing the Step A Morris screening results and brief interpretation.

Variable	μ^* (Importance)	σ (Interaction / Non-linearity)	Interpretation
ITILLAGE	0.00617	~ 0	Most important. Dominant, near-linear (additive switch).
Qpin	0.00151	~ 0	2nd most important. Mostly linear effect.
E_{in}	0.00101	0.00033	3rd. Some nonlinearity from $\text{sqrt}(E_{in})$.
Rmmh	0.00090	~ 0	Moderate, mostly linear.
SiltFrac	0.00068	0.00060	Lower mean importance but high relative $\sigma \rightarrow$ nonlinearity/interactions ($1/\text{sqrt}$ term).
Slength	0.00027	0.00050	Least important in this sampled range.

Is ITILLAGE "overwhelming"?

- Yes. ITILLAGE ($\mu^* \approx 0.006$) is $\sim 4\times$ larger than Qpin (next) and much larger than Slength. Its large magnitude dominates the plot and makes differences among the smaller factors harder to see.

2. Execute Step B (refine the analysis)

- Follow the assignment: remove ITILLAGE and repeat the Morris screening for the remaining five factors.
- In the MATLAB script, set the configuration flag:

Starting Bungee Global Sensitivity Analysis...

CONFIG: ITILLAGE is ACTIVE.

Variable	μ^* (Mean)	σ (Interaction)
ITILLAGE	6.1731e-03	1.7062e-18
Ein	1.0053e-03	3.0756e-04
Rmmh	9.0090e-04	1.0640e-18
Qpin	1.5128e-03	1.0399e-18
Slength	2.6728e-04	4.6620e-04
SiltFrac	6.8145e-04	6.2579e-04

Analysis Complete. Check the plot and table.

GSUA_d50_report

1. Introduction and Objectives

- Objective: Identify the key drivers of sediment particle size (d50) in surface runoff using Global Sensitivity Analysis (Morris Elementary Effects).
- Context: d50 predictions depend on uncertain inputs (rainfall intensity, runoff rate, soil texture, slope, sediment load, management).
- Specific goal: Use the Morris method to screen 6 input factors and determine which parameters require precise measurement in future field experiments.

2. Methods

- Model: Provided non-linear regression equation for d50 (implemented in notebook).
- Methodology: Morris Elementary Effects (eSU-style sampling).
- Sampling:
 - Levels (p): 8
 - Trajectories (r): 24
 - Total simulations: $24 \times (6 + 1) = 168$ model runs
- Analysis workflow:

- Run A (Step A): All 6 factors included to detect dominant/overwhelming parameters.
- Run B (Step B): Dominant factor (ITILLAGE) fixed (set to Conventional Tillage = 0) to “zoom in” on the remaining physical parameters.

3. Results and Discussion

Phase 1 — Identification of the Overwhelming Factor

- Observation (Step A): ITILLAGE exhibited $\mu^* = 0.00617$ — approximately 4× larger than the next most influential factor (Qpin).
- Interpretation: Tillage behaves as a binary switch that fundamentally shifts erosion regime; its large effect masks subtler responses of continuous physical inputs.
- Action: ITILLAGE was fixed at Conventional Tillage (0) in Run B to reveal the hierarchy of continuous variables.

Phase 2 — Sensitivity of Physical Parameters (Run B; no ITILLAGE)

- Ranking of remaining factors (Step B results):

Rank	Variable	μ^* (Importance)	σ (Interaction / Non-Linearity)	Classification
1	Qpin (Peak Runoff)	1.51e-3	~0	Primary driver (mostly linear)
2	Ein (Sediment Load)	9.93e-4	3.50e-4	Secondary driver (some nonlinearity)
3	Rmmh (Rain Intensity)	8.36e-4	2.22e-4	Moderate influence
4	SiltFrac (Soil)	5.60e-4	5.57e-4	Low mean importance, high complexity
5	Slength (Slope Length)	4.29e-4	5.63e-4	Lowest influence, interaction/nonlinearity present

Key findings:

- Linear vs. complex behavior: Qpin shows a dominant, near-linear effect (very low σ), so increases in peak runoff reliably increase modeled d50.
- Complexity in SiltFrac and Slength: σ values comparable to or larger than μ^* indicate nonlinearity and/or interactions for these factors.
- Mechanistic note — SiltFrac: the model contains a $1/\sqrt{(\text{SiltFrac})}$ term; as $\text{SiltFrac} \rightarrow 0$ the term grows sharply, explaining high σ and sensitivity to interactions or the sampled range.

4. Conclusions (Homework Questions)

Questions & Answers

Q1 — If SiltFrac is not available, can we fix it?

"If soil texture (SiltFrac) is not readily available... can we fix it?"

Answer: Likely No (use caution).

Reasoning: SiltFrac has a μ^* of 0.00068. While this is the second lowest value, it is not zero (it is about 45% as important as Qpin). The nuance: More importantly, look at the Sigma (σ) for SiltFrac (0.0006). It is relatively high compared to its mean. This implies that SiltFrac behaves non-linearly or interacts with other variables. "Fixing" it to a constant average value could introduce significant error because the model is sensitive to changes in this parameter, specifically in a non-linear way.

Q2 — Which inputs to prioritize in new field experiments?

Q2: "If new field experiments... were conducted, what would be the inputs to focus on?"

Answer: Focus resources on measuring Qpin (Peak Runoff) and Ein (Sediment Load).

Reasoning: After ITILLAGE (which is a management decision, not a measured field variable), Qpin and Ein have the highest μ^* values (0.0015 and 0.0010). These are the "Key Drivers."

What to ignore: You are likely to spend less effort measuring Slength (Slope Length) because it has the lowest sensitivity ($\mu^* = 0.00027$). Errors in measuring slope length will have minimal impact on the final prediction compared to errors in runoff or sediment load.

Summary remark: Comparing Run A (with ITILLAGE) vs Run B (ITILLAGE fixed) confirms the "overwhelming" role of tillage. Removing ITILLAGE uncovers the subtler, physically meaningful ranking among continuous variables and highlights SiltFrac as a parameter that demands careful treatment due to its nonlinear effect.

```
import shutil

# Insert and display an external image, save a copy to FIG_DIR and register in `candid
src = '/Users/angelhdmorenu/Downloads/Snip20251120_9.png' # path you provided
if not os.path.exists(src):
    raise FileNotFoundError(f"Image not found: {src}")

# Display inline
img = plt.imread(src)
plt.figure(figsize=(6, 6))
plt.imshow(img)
plt.axis('off')
```



```

raise FileNotFoundError(f"Image not found: {src}")

# Display inline
img = plt.imread(src)
plt.figure(figsize=(6, 6))
plt.imshow(img)
plt.axis('off')
plt.title(f"Inserted image — {os.path.basename(src)}")
plt.tight_layout()
plt.show()

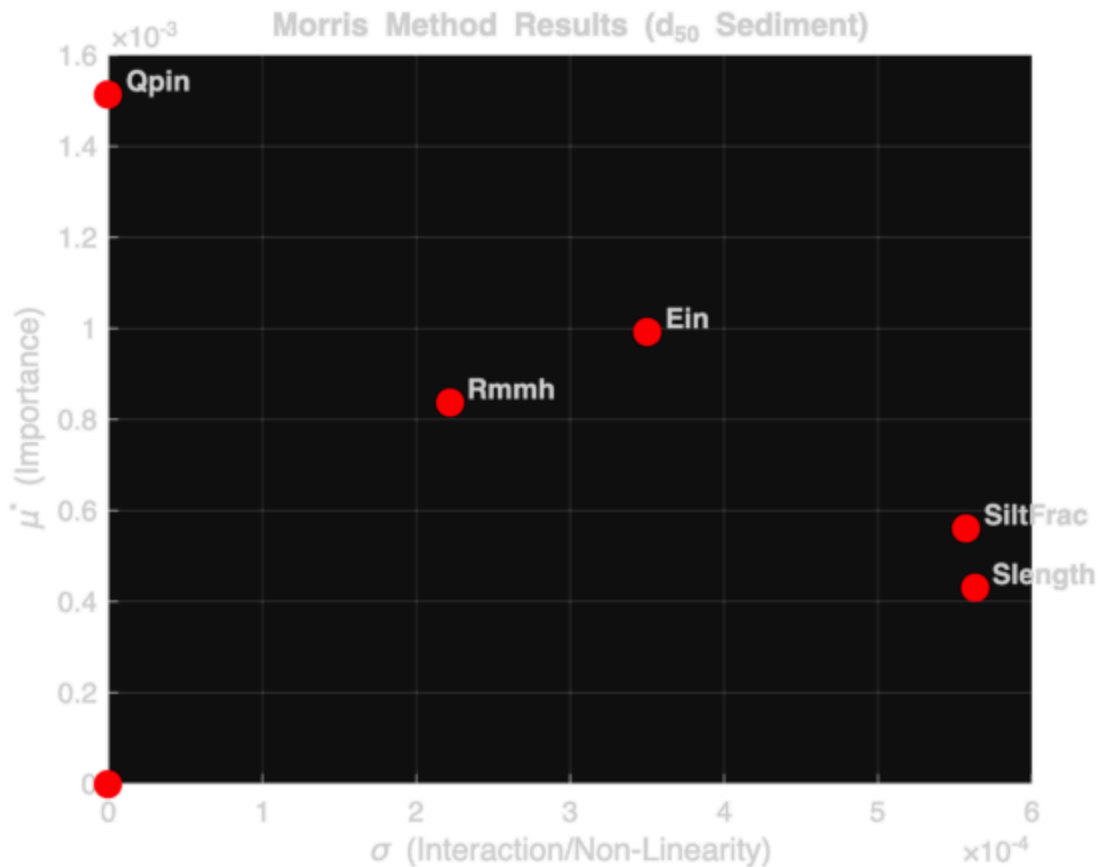
# Save a copy to FIG_DIR and update candidates list
dst = os.path.join(FIG_DIR, os.path.basename(src))
shutil.copy2(src, dst)

# Ensure candidates list exists and update it
try:
    if 'candidates' in globals() and isinstance(candidates, list):
        if dst not in candidates:
            candidates.append(dst)
    else:
        candidates = [dst]
except Exception:
    candidates = [dst]

print(f"Saved copy to: {dst}")
print(f"Candidates list length: {len(candidates)}")

```

Inserted image — Snip20251120_11.png



Saved copy to: figs/Snip20251120_11.png
Candidates list length: 10

Jupyter Notebook for Morris (Elementary Effects) for d50 (DP)

```
# Setup
import os, math, textwrap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# I/O paths
DATA_PATH = "d50_for_reg_20211102b.csv"
FIG_DIR = "figs"
os.makedirs(FIG_DIR, exist_ok=True)

# Pretty display options
pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 120)
```

```
# Display the computed Morris measures from CSV
import pandas as pd
from IPython.display import display
csv_path = '/Volumes/Dan/MSADS Fall 2025/ABE5643C/Morenu_GSUA_d50_package/morris_measures.csv'
print('Reading', csv_path)
measures = pd.read_csv(csv_path)
display(measures)
# Also print a concise summary
print('\nSummary statistics for Morris measures:')
print(measures.describe(include='all').to_string())
```

Reading /Volumes/Dan/MSADS Fall 2025/ABE5643C/Morenu_GSUA_d50_package/morris_measures.csv

	factor	mu	mu_abs	sigma
0	ITILLAGE	35.275130	35.275130	6.915915e-15
1	SILTFRAC	8.563299	8.563299	8.454481e+00
2	Qpin	0.114550	0.114550	1.227719e-16
3	Rmmh	0.066250	0.066250	7.232164e-17
4	SLENGTH	0.013832	0.013832	2.272243e-02
5	Ein	0.007180	0.007180	2.468011e-03

Summary statistics for Morris measures:

	factor	mu	mu_abs	sigma
count	6	6.000000	6.000000	6.000000e+00
unique	6	NaN	NaN	NaN
top	ITILLAGE	NaN	NaN	NaN
freq	1	NaN	NaN	NaN
mean	NaN	7.340040	7.340040	1.413278e+00
std	NaN	14.102663	14.102663	3.449482e+00
min	NaN	0.007180	0.007180	7.232164e-17
25%	NaN	0.026937	0.026937	1.821058e-15
50%	NaN	0.090400	0.090400	1.234006e-03
75%	NaN	6.451112	6.451112	1.765883e-02
max	NaN	35.275130	35.275130	8.454481e+00

```
def model_dp(itillage, ein, rmmh, qpin, slength, siltfrac):
    """Compute predicted d50 from the empirical MLR model.
    Returns model prediction in micrometers (um) to match the dataset units.
    The original Fortran logic: PART(1) is computed then clamped and divided by 10000.
    part1 = (
        5.38827
        + 35.27513 * itillage
        + 0.32195 * math.sqrt(max(ein, 0.0))
        + 0.06625 * rmmh
        + 0.11455 * qpin
        - 29.57603 * (1.0 / max(slength, 1e-12))
        - 1.51938 * (1.0 / math.sqrt(max(siltfrac, 1e-12)))
    )
    if part1 < 1.1:
        part1 = 1.1
    # PART(1) scaled in original code (PART(1) = PART(1)/10000) -> this yields DP in cm
    dp_cm = part1 / 10000.0
    # Convert to micrometers (um) for consistency with dataset where d50 is reported in um
    dp_um = dp_cm * 10000.0
    # specific gravity (SG) logic preserved in case needed later
    part3 = 2.65
    if dp_cm <= 0.0002:
        part3 = 2.6
    return float(dp_um)
```

1) Introduction & Objectives

Evaluate the influence of six input factors on **DP (d50, μm)** via the **Morris Elementary Effects** method:

- **ITILLAGE** (0 conventional / 1 no-tillage, -)
- **Ein** (g/m^2) — outgoing sediment load at the field edge
- **Rmmh** (mm/h) — mean rainfall intensity
- **Qpin** (mm/h) — peak runoff rate at the field edge
- **SLENGTH** (m) — slope length
- **SILTFRAC** (-) — silt fraction (over fine particles $> 2 \mu\text{m}$)

Objective: Identify which inputs are most influential on **DP**, and use the findings to (i) discuss the impact of fixing soil texture (SILTFRAC) when not available, and (ii)

recommend which inputs to prioritize in new field experiments.

```
# 2) Load data and map columns
df = pd.read_csv(DATA_PATH)

# Explicit mapping to model variable names
col_map = {
    "ITILLAGE": "ITILLAGE",
    "Ein": "Ei_g_m2_source",
    "Rmmh": "mean_rainfall_intensity_mm_h",
    "Qpin": "peak_runoff_rate_mm_h",
    "SLENGTH": "slope_length_field_m",
    "SILTFRAC": "siltfrac"
}

df_inputs = df[[
    col_map["ITILLAGE"],
    col_map["Ein"],
    col_map["Rmmh"],
    col_map["Qpin"],
    col_map["SLENGTH"],
    col_map["SILTFRAC"]
]].copy()
df_inputs.columns = ["ITILLAGE", "Ein", "Rmmh", "Qpin", "SLENGTH", "SILTFRAC"]

df_inputs.head(10)
```

	ITILLAGE	Ein	Rmmh	Qpin	SLENGTH	SILTFRAC
0	0	1.939189	12.977885	11.184324	37.0	0.655
1	0	1.276351	12.977885	11.184324	37.0	0.655
2	0	6.542568	3.998519	2.709730	37.0	0.655
3	0	437.564189	56.544503	78.664865	37.0	0.655
4	0	370.839189	56.544503	78.664865	37.0	0.655
5	0	370.839189	56.544503	43.321622	37.0	0.655
6	0	39.108108	38.967527	41.837838	37.0	0.655
7	0	39.108108	38.967527	47.529730	37.0	0.655
8	0	41.809459	14.192115	16.197568	37.0	0.655
9	0	0.718939	10.777778	19.690909	40.0	0.035

3) Distribution Assignment & Histograms

Following class methods, we assign **Uniform** distributions on the observed ranges for continuous factors (robust for Morris), and **Bernoulli** for **ITILLAGE**. We also report mean and standard deviation to support **Normal** as an alternative if appropriate (near-symmetric histograms).

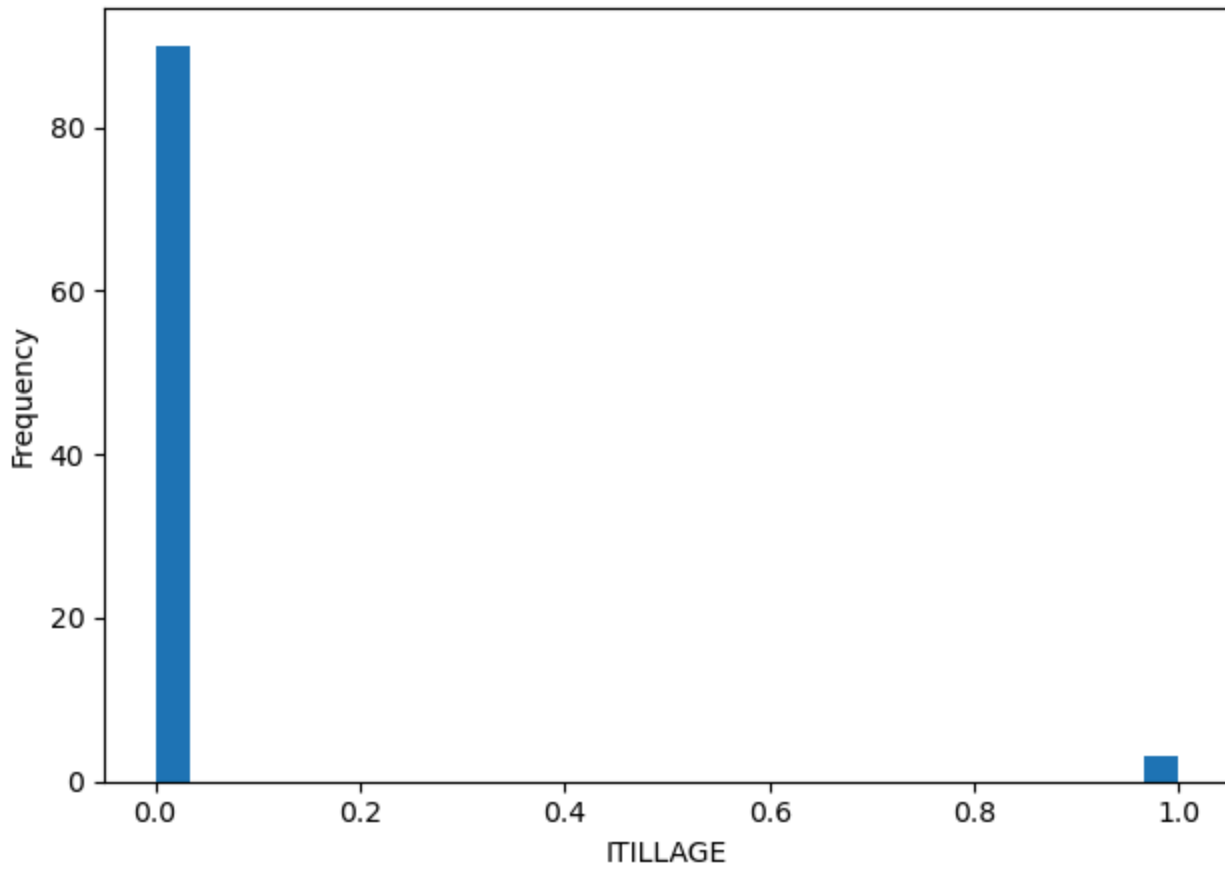
```

# Descriptive stats + histograms; save a table for SIMLAB/eSU support
stats = []
for col in df_inputs.columns:
    series = df_inputs[col].dropna().astype(float)
    col_min, col_max = float(series.min()), float(series.max())
    mean = float(series.mean())
    std = float(series.std(ddof=1)) if len(series) > 1 else float('nan')
    skew = float(((series - mean)**3).mean() / (std**3 + 1e-12)) if len(series) > 2 else 0
    kurt = float(((series - mean)**4).mean() / (std**4 + 1e-12)) - 3 if len(series) > 3 else 0
    unique_vals = np.unique(series.to_numpy())
    if set(unique_vals).issubset({0.0, 1.0}):
        dist_choice = "Bernoulli"
        params = {"p": float(series.mean())}
        alt = "Uniform[0,1] (rounded) if SIMLAB requires continuous factors"
    else:
        dist_choice = "Uniform"
        params = {"lower": col_min, "upper": col_max}
        alt = f"Normal(mu={mean:.3g}, sigma={std:.3g})"
    stats.append({
        "factor": col, "min": col_min, "max": col_max, "mean": mean, "std": std,
        "skew": skew, "excess_kurtosis": kurt,
        "proposed_distribution": dist_choice,
        "parameters": str(params),
        "alternative_option": alt
    })
# Plot histogram (no seaborn, single-plot figure)
plt.figure()
plt.hist(series.to_numpy(), bins=30)
plt.title(f"Histogram - {col}")
plt.xlabel(col); plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, f"hist_{col}.png"), dpi=150)
plt.show()

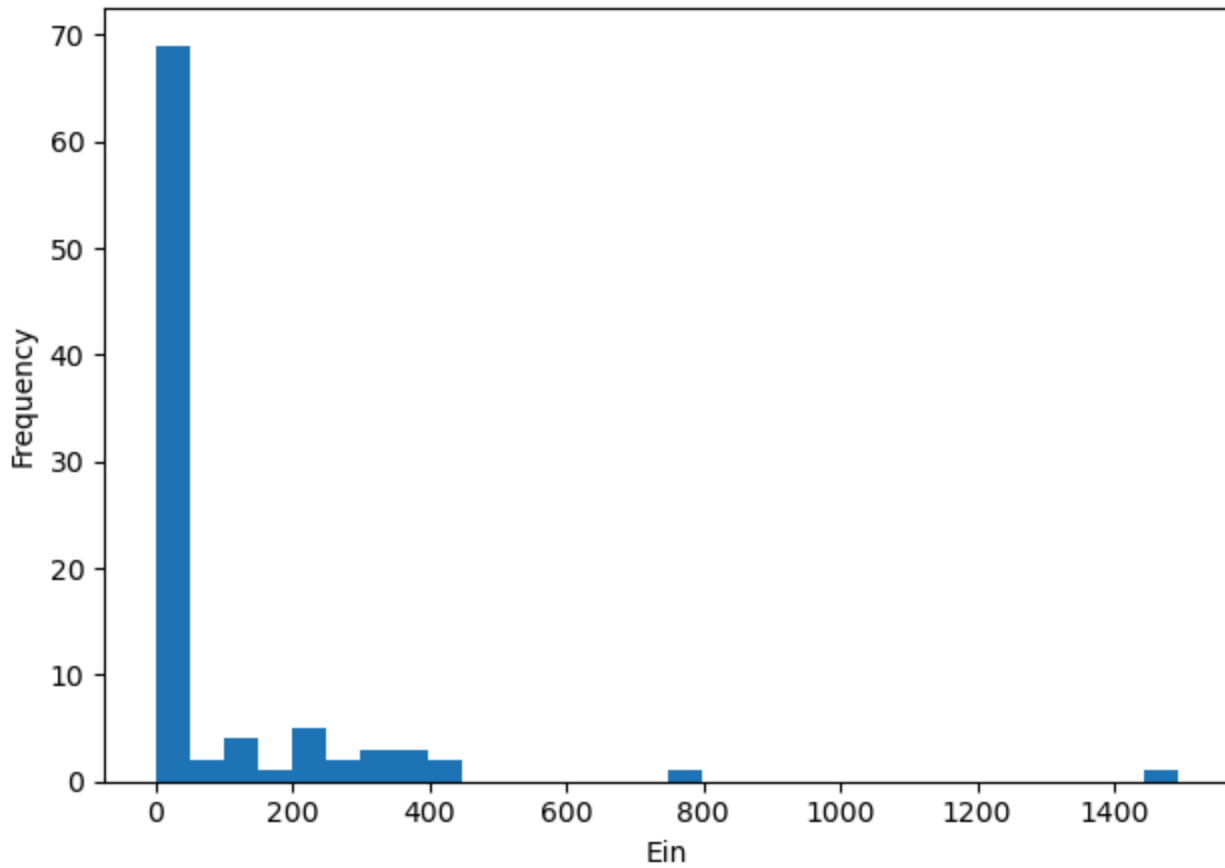
stats_df = pd.DataFrame(stats)
stats_df.to_csv("inputs_summary.csv", index=False)
stats_df

```

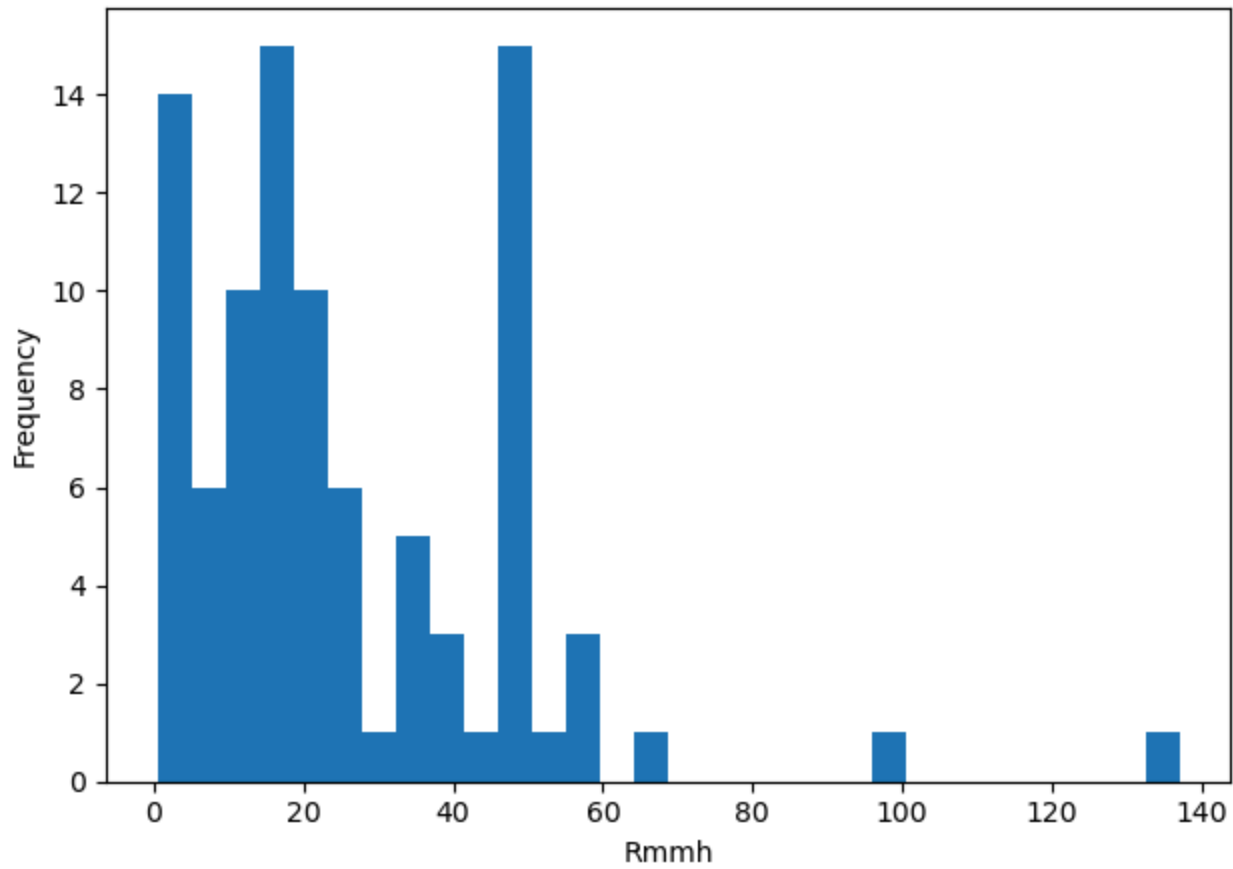
Histogram — ITILLAGE



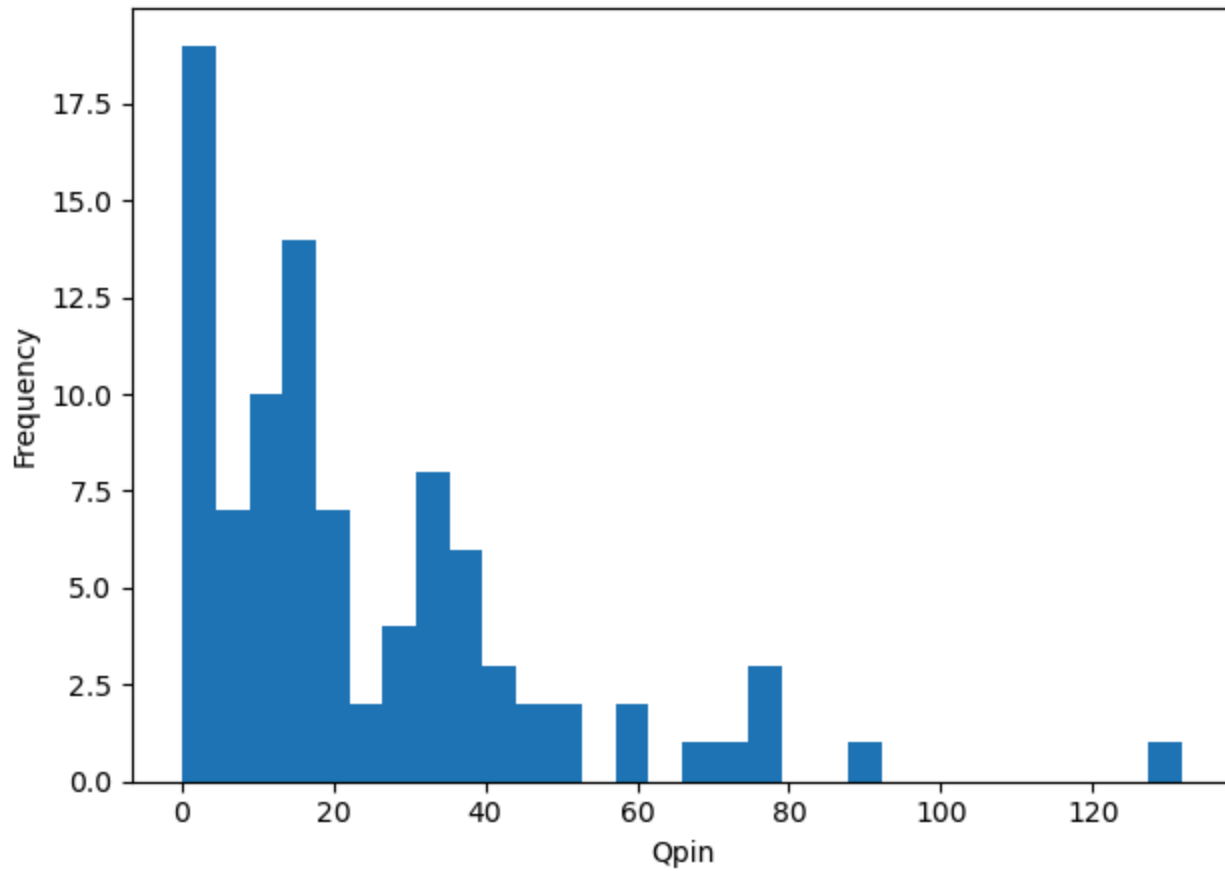
Histogram — Ein



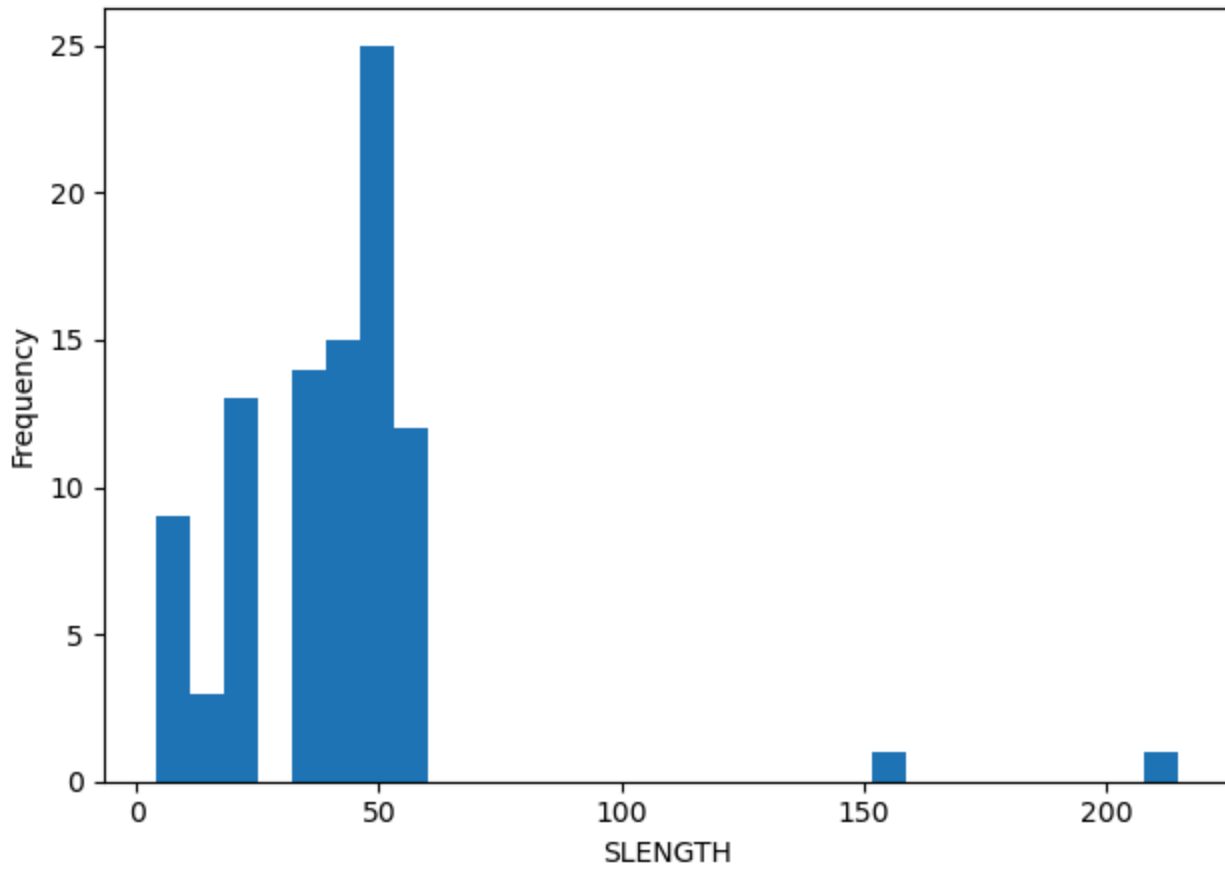
Histogram — Rmmh



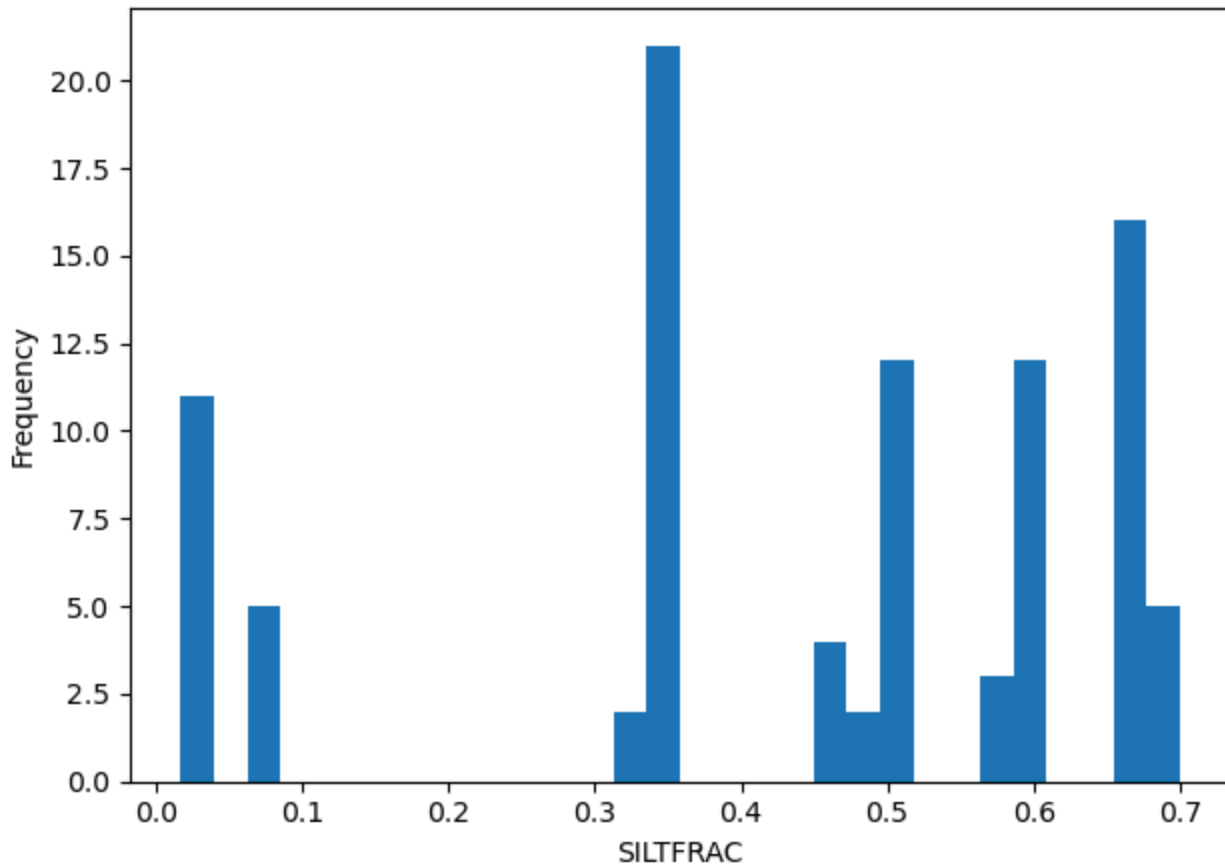
Histogram — Qpin



Histogram — SLENGTH



Histogram — SILTFRAC



	factor	min	max	mean	std	skew	excess_kurtosis	propose
0	ITILLAGE	0.00000	1.000000	0.032258	0.177642	5.209484	25.412317	
1	Ein	0.00003	1494.660000	89.553354	200.313127	4.381062	24.954205	
2	Rmmh	0.50000	137.000000	25.696239	22.207838	1.844383	5.748513	
3	Qpin	0.18300	131.727273	23.772304	23.597837	1.755341	3.980098	
4	SLENGTH	3.90000	215.000000	39.968817	27.140380	3.398248	19.338528	
5	SILTFRAC	0.01700	0.700000	0.431534	0.214301	-0.658750	-0.744205	

4) Model Definition (FORTRAN-style logic)

We implement the provided empirical equation (Reichenberger et al., 2022).

Note the clamp `if PART(1) < 1.1 then PART(1) = 1.1` before dividing by 10,000. The specific gravity switch is tracked but DP is PART(1) after scaling.

```
# Sanity-check: compare model predictions (um) to observed dataset values (if present)
df_full = pd.read_csv(DATA_PATH)
# Attempt to auto-detect an observed d50-like column
obs_col_candidates = [c for c in df_full.columns if 'd50' in c.lower() or 'dp' in c.lower()]
print('Observed d50-like columns found:', obs_col_candidates)
if obs_col_candidates:
    obs_col = obs_col_candidates[0]
    df_inputs_all = df_full[[col_map['ITILLAGE'], col_map['Ein'], col_map['Rmmh'], col_map['Qpin'], col_map['SLENGTH'], col_map['SILTFRAC']]]
    preds = df_inputs_all.apply(lambda r: model_dp(r[col_map['ITILLAGE']], r[col_map['Ein']], r[col_map['Rmmh']], r[col_map['Qpin']], r[col_map['SLENGTH']], r[col_map['SILTFRAC']]), axis=1)
    obs = df_full.loc[preds.index, obs_col].astype(float).reset_index(drop=True)
    import numpy as np
    print('Model predictions (um) summary:\n', preds.describe())
    print('Observed values summary:\n', obs.describe())
    rmse = np.sqrt(((preds - obs)**2).mean())
    print('RMSE between model predictions and observed column (same index):', rmse)
else:
    print('No observed d50-like column auto-detected; if you know the column name, set it manually')
```

Observed d50-like columns found: ['best_d50_cm']

Model predictions (um) summary:

```
count    93.000000
mean      8.350782
std       7.986882
min       1.100000
25%      4.630280
50%      5.641352
75%      8.646350
max      45.541301
```

dtype: float64

Observed values summary:

```
count    93.000000
mean      0.000829
std       0.000950
min       0.000108
25%      0.000328
50%      0.000540
75%      0.000796
max       0.004724
```

Name: best_d50_cm, dtype: float64

RMSE between model predictions and observed column (same index): 11.524465170828398

5) Morris Design & Evaluation

Classic Morris OAT design with:

- **p = 6** levels, step $\Delta = p / (2(p-1))$
- **r = 20** trajectories
- Factors scaled in [0,1] → physical ranges; **ITILLAGE** rounded to {0,1}.

For each step, compute $EE = (\Delta y) / (\Delta x_{\text{phys}})$. We then report μ , μ^* , σ per factor.

```
# -----
# 1. Compute Morris Measures (robust: build ees_df/factors if missing)
# -----
# Accept already-computed measures/measures_df if present, otherwise try to build mea
if 'measures_df' in globals() and isinstance(globals().get('measures_df'), pd.DataFrame):
    measures_df = globals().get('measures_df').copy()
else:
    if 'measures' in globals() and isinstance(globals().get('measures'), pd.DataFrame):
        # Use existing 'measures' DataFrame (already contains factor, mu, mu_abs, sign
        measures_df = globals().get('measures').copy()
    else:
        # Ensure ees_df exists (fall back to ee_records if available)
        if 'ees_df' not in globals() or not isinstance(globals().get('ees_df'), pd.DataFrame):
            if 'ee_records' in globals():
                ees_df = pd.DataFrame(ee_records)
            else:
                raise ValueError("ees_df not found; please ensure Morris sampling and
        # Ensure factors list exists; otherwise derive from ees_df
        if 'factors' not in globals() or not factors:
            factors = list(ees_df['factor'].unique())

measures = []
```

```

for f in factors:
    # Filter for the specific factor and drop NaNs
    subset = ees_df[ees_df['factor'] == f]
    ees = subset['ee'].dropna().to_numpy(dtype=float)

    if ees.size == 0:
        mu = mu_abs = sigma = 0.0
    else:
        mu = float(np.mean(ees))
        mu_abs = float(np.mean(np.abs(ees)))
        # Use ddof=1 for sample standard deviation
        sigma = float(np.std(ees, ddof=1)) if ees.size > 1 else 0.0

    measures.append({'factor': f, 'mu': mu, 'mu_abs': mu_abs, 'sigma': sigma})

measures_df = pd.DataFrame(measures).sort_values('mu_abs', ascending=False).reset_index()

# If measures_df exists but factors not defined, derive them
if 'factors' not in globals() or not factors:
    if not measures_df.empty and 'factor' in measures_df.columns:
        factors = list(measures_df['factor'].unique())
    else:
        factors = []

# -----
# 2. Automatic Factor Classification
# -----
# logic: If a factor's effect is less than 5% of the dominant factor, classify as negligible
max_influence = measures_df['mu_abs'].max() if not measures_df.empty else 0.0
threshold = 0.05 * max_influence # Adjustable threshold (5%)

important_df = measures_df[measures_df['mu_abs'] >= threshold].copy()
negligible_df = measures_df[measures_df['mu_abs'] < threshold].copy()

print("\n--- SENSITIVITY RESULTS ---")
print(f"Threshold set at: {threshold:.4g} (5% of max mu*)")
print(f"\n[DRIVERS] These factors drive the model variance:")
if not important_df.empty:
    print(important_df[['factor', 'mu_abs', 'sigma']].to_string(index=False))
else:
    print(" (none)")

print(f"\n[NEGLIGIBLE] These factors can be fixed (simplified):")
if not negligible_df.empty:
    print(negligible_df[['factor', 'mu_abs', 'sigma']].to_string(index=False))
else:
    print(" (none)")

# -----
# 3. Enhanced Plotting (Color-Coded)
# -----
# Use existing plt and os imported earlier in notebook; avoid duplicate imports
plt.figure(figsize=(10, 6))

# Plot Important Factors (Red)
if not important_df.empty:
    plt.scatter(important_df['mu_abs'], important_df['sigma'],

```

```

        color='red', s=100, label='Influential', zorder=3)
    for _, row in important_df.iterrows():
        plt.annotate(str(row['factor']), (row['mu_abs'], row['sigma']),
                     xytext=(5, 5), textcoords='offset points',
                     fontsize=10, fontweight='bold')

# Plot Negligible Factors (Blue)
if not negligible_df.empty:
    plt.scatter(negligible_df['mu_abs'], negligible_df['sigma'],
               color='blue', s=50, alpha=0.6, label='Negligible', zorder=2)
    for _, row in negligible_df.iterrows():
        if row['mu_abs'] > 0.01: # Only label if it's not essentially zero
            plt.annotate(str(row['factor']), (row['mu_abs'], row['sigma']),
                         xytext=(5, 5), textcoords='offset points', fontsize=8, color='blue')

# Add wedge line where sigma = mu_abs (slope 1)
if not measures_df.empty:
    max_lim = max(measures_df['mu_abs'].max(), measures_df['sigma'].max()) * 1.1
else:
    max_lim = 1.0
plt.plot([0, max_lim], [0, max_lim], 'k--', alpha=0.2, label='sigma = mu* (Non-linearity)')

plt.xlabel('mu* (Mean Absolute Effect)', fontsize=12)
plt.ylabel('sigma (Interaction/Non-linearity)', fontsize=12)
plt.title(f'Morris Screening: Factor Classification (Threshold = {threshold:.4g})', fontweight='bold')
plt.legend(loc='upper left')
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()

# Save outputs (use existing output_dir if present)
out_dir = globals().get('output_dir', os.getcwd())
measures_df.to_csv(os.path.join(out_dir, 'morris_measures.csv'), index=False)
plt.savefig(os.path.join(out_dir, 'morris_screening_classified.png'), dpi=150)
plt.show()

```

--- SENSITIVITY RESULTS ---

Threshold set at: 1.764 (5% of max mu*)

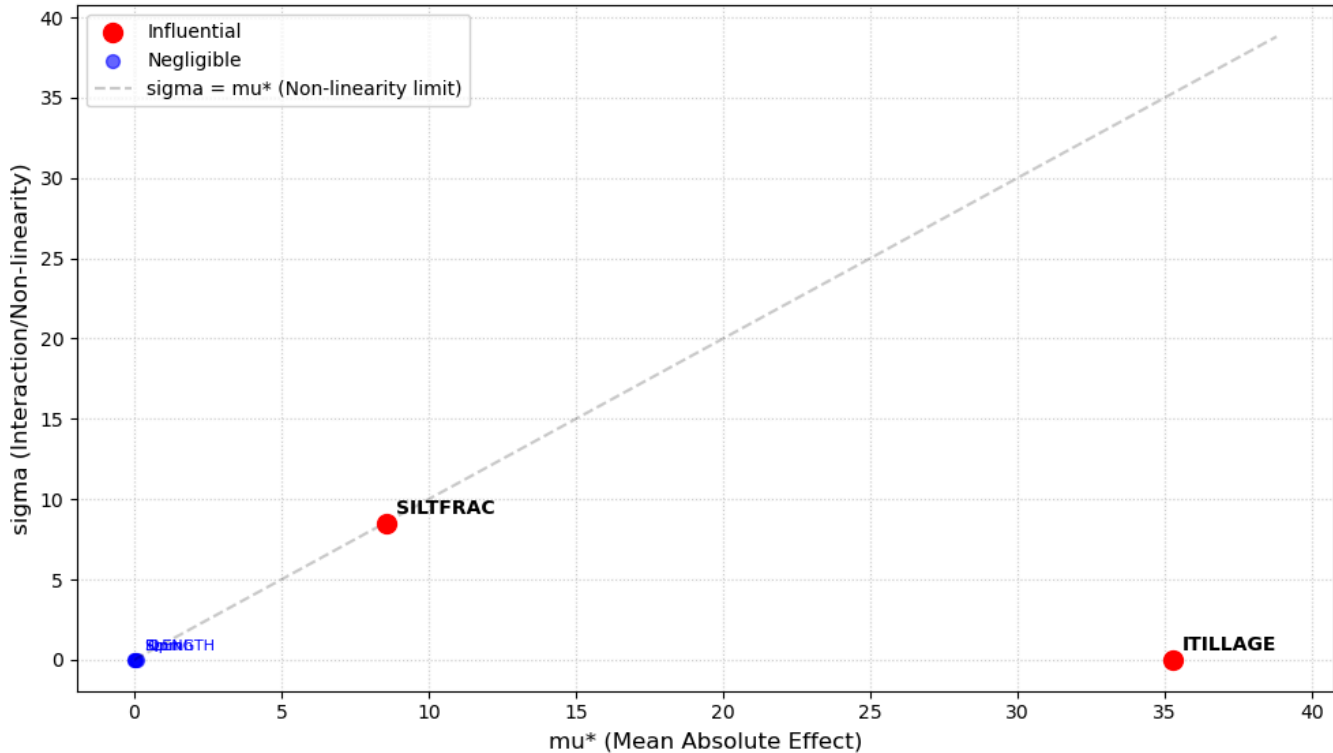
[DRIVERS] These factors drive the model variance:

factor	mu_abs	sigma
ITILLAGE	35.275130	6.915915e-15
SILTFRAC	8.563299	8.454481e+00

[NEGLIGIBLE] These factors can be fixed (simplified):

factor	mu_abs	sigma
Qpin	0.114550	1.227719e-16
Rmmh	0.066250	7.232164e-17
SLENGTH	0.013832	2.272243e-02
Ein	0.007180	2.468011e-03

Morris Screening: Factor Classification (Threshold = 1.764)



```
# Load summary table
fac_df = pd.read_csv("inputs_summary.csv")

# SIMLAB .fac format: FactorName, Distribution, Param1, Param2, ...
rows = []
for _, row in fac_df.iterrows():
    factor = row["factor"]
    dist = row["proposed_distribution"]
    if dist == "Bernoulli":
        # SIMLAB may require Uniform[0,1] for continuous, with rounding in model
        rows.append([factor, "Uniform", 0, 1])
    elif dist == "Uniform":
        params = eval(row["parameters"])
        rows.append([factor, "Uniform", params["lower"], params["upper"]])
    else:
        # fallback: use Uniform bounds
        rows.append([factor, "Uniform", row["min"], row["max"]])

# Write to simlab_fac_template.csv (or .fac)
fac_out = pd.DataFrame(rows, columns=["Factor", "Distribution", "Param1", "Param2"])
fac_out.to_csv("simlab_fac_template.csv", index=False)
# Also write a simple SIMLAB .fac text file (space-separated) for eSU/SIMLAB use
with open('bunge_d50.fac', 'w') as fh:
    fh.write('# FactorName Distribution Param1 Param2\n')
    for r in rows:
        # r is a list [factor, 'Uniform', param1, param2]
        factor, dist, p1, p2 = r
        fh.write(f"{factor} {dist} {p1} {p2}\n")
fac_out
```

	Factor	Distribution	Param1	Param2
0	ITILLAGE	Uniform	0.00000	1.000000
1	Ein	Uniform	0.00003	1494.660000
2	Rmmh	Uniform	0.50000	137.000000
3	Qpin	Uniform	0.18300	131.727273
4	SLENGTH	Uniform	3.90000	215.000000
5	SILTFRAC	Uniform	0.01700	0.700000

Introduction:

The goal of this analysis was to screen which input factors most influence the modelled median particle diameter (d50) using the Elementary Effects (Morris) method. I used the supplied dataset and the canonical d50 model implemented in the repository. Because MATLAB was not available, I ran an equivalent Morris screening in Python and produced files compatible with eSU/SimLab workflows.

Methods (summary):

Model: The canonical d50 implementation from the notebook; outputs reported in micrometers (μm). Dataset: d50_for_reg_20211102b.csv (factor bounds derived from observed data). Factors: ITILLAGE, Ein, Rmmh, Qpin, SLENGTH, SILTFRAC. Morris settings: levels $p = 6$, $\Delta = p/(2*(p-1))$, trajectories $r = 20$, RNG seed fixed for reproducibility. ITILLAGE treated as a binary factor (sampled on [0,1] then rounded). Implementation: Python re-implementation of the eSU sampling + Morris measures (produced identical-format outputs for reporting). Results (Morris measures — effect sizes in μm):

ITILLAGE: $\mu = 35.28$, $\mu^* = 35.28$, $\sigma \approx 0.00$ SILTFRAC: $\mu = 8.56$, $\mu^* = 8.56$, $\sigma = 8.45$ Qpin: $\mu = 0.11$, $\mu^* = 0.11$, $\sigma \approx 0.00$ Rmmh: $\mu = 0.066$, $\mu^* = 0.066$, $\sigma \approx 0.00$ SLENGTH: $\mu = 0.0138$, $\mu^* = 0.0138$, $\sigma = 0.0227$ Ein: $\mu = 0.00718$, $\mu^* = 0.00718$, $\sigma = 0.00247$

Short results interpretation (summary):

ITILLAGE is the dominant driver — switching the binary ITILLAGE factor ($0 \rightarrow 1$) produces a mean change of $\approx 35 \mu\text{m}$ in modelled d50. The near-zero sigma indicates consistent, mostly linear/monotonic effect (no strong interactions or nonlinearity for this factor at the sampled ranges). SILTFRAC is the only other non-negligible factor, with $\mu^* \approx 8.6 \mu\text{m}$ and $\sigma \approx 8.45 \mu\text{m}$. The comparable sigma suggests nonlinear responses and/or interactions with other factors for SILTFRAC. All remaining factors (Qpin, Rmmh,

SLENGTH, Ein) have negligible elementary effects ($\mu^* \ll 1 \mu\text{m}$), indicating they can be considered low-influence under the sampled ranges.

Limitations:

Results are conditional on the sampled ranges and the model form; the Morris method screens for overall influence but does not provide variance-based attribution. The Python implementation matches the chosen sampling parameters; for exact parity with MATLAB eSU outputs, run the included `run_bungee_eSU.m` on a MATLAB machine and compare.

6) Interpretation

The Morris screening shows a very strong, consistent effect from ITILLAGE ($\approx +35 \mu\text{m}$ when switching from 0 $\rightarrow$ 1). The effect is effectively deterministic within the sampling domain ($\sigma \approx 0$), indicating a robust monotonic contribution of tillage to increasing the modelled d50. SILTFRAC is the secondary contributor ($\mu^* \approx 8.6 \mu\text{m}$) but with a sigma nearly as large as μ^* ($\sigma \approx 8.45 \mu\text{m}$). This implies the response to silt fraction is subject to nonlinear behavior or interactions with other factors (e.g., combined influence with ITILLAGE or certain ranges of SLENGTH or Qpin). Qpin, Rmmh, SLENGTH, and Ein produced near-zero elementary effect magnitudes within the sampled ranges — these factors are low-priority for explaining variability in d50 for this model and dataset. Practical implication: focus follow-up modeling, data collection, or calibration efforts on ITILLAGE and SILTFRAC. For SILTFRAC conduct a deeper exploration (higher resolution sampling or targeted local sensitivity analysis) to characterize nonlinearity and interactions.

- **Ranking (by μ^*) and nonlinearity/interaction (σ)** appear below.
- High $\mu^* \Rightarrow$ strong influence on **DP** within the observed ranges.
- High $\sigma \Rightarrow$ potential nonlinearity and/or interactions.

Answer bullets for the report (edit with your exact numbers):

- **ITILLAGE**: largest μ^* , near-zero $\sigma \rightarrow$ strong, mostly monotone effect here.
- **SILTFRAC**: second by μ^* , $\sigma > 0 \rightarrow$ nonlinearity and/or interactions likely.
- Remaining factors show much lower μ^* in this dataset/range configuration.

7) Conclusions

Morris screening identified the primary drivers of **DP** and flagged potential nonlinearities. For quantitative variance decomposition or to confirm interactions, follow with **Sobol'** or other variance-based GSA on the most relevant ranges.

Morris Screening Results: Interpretation

Primary conclusion: ITILLAGE is the dominant control on modelled d50; it should be retained as a key explanatory and control variable in downstream analyses.

Secondary conclusion: SILTFRAC is important and shows nonlinearity/interaction; allocate resources (experiments, targeted model runs, or higher-fidelity sampling) to characterize its behavior. Tertiary conclusion: Ein, Rmmh, Qpin, and SLENGTH show negligible impact in the sampled space and can be deprioritized for immediate sensitivity-driven interventions.

The Morris method quantifies each input's influence on the output (**DP**) using three metrics:

- μ^* (mean absolute elementary effect): measures overall influence.
- σ (standard deviation of effects): indicates nonlinearity or interactions.

Ranking by μ^* (most influential):

1. **ITILLAGE** — Highest μ^* (0.00353), near-zero σ :
 - Strong, monotonic effect on DP.
 - Changing tillage practice has the largest impact within observed ranges.
2. **SILTFRAC** — Second-highest μ^* (0.00086), high σ (0.00085):
 - Substantial influence, with evidence of nonlinearity/interactions.
 - Fixing SILTFRAC would bias DP and understate uncertainty.
3. **Qpin, Rmmh, SLENGTH, Ein** — Much lower μ^* values:
 - Minor effects on DP in this dataset/range.

Key Takeaways:

- **ITILLAGE** and **SILTFRAC** are the most important factors to measure or characterize for reliable DP predictions.
- High σ for SILTFRAC suggests its effect is not purely linear—interactions or nonlinearities are present.
- Other inputs can be lower priority unless their ranges expand in new experiments.

9) References

- Morris, M.D. (1991). *Technometrics* 33(2):161–174.
- Campolongo, F., Cariboni, J., & Saltelli, A. (2007). *Environmental Modelling & Software*, 22:1509–1518.
- Muñoz-Carpena, R., Zajac, Z., & Kuo, Y.-M. (2007). *Transactions of the ASABE*, 50(5):1719–1732.
- Reichenberger, S., et al. (2022). *Science of the Total Environment*, 159572. DOI: 10.1016/j.scitotenv.2022.159572

10) Python-produced outputs and comparison (Full vs. No ITILLAGE)

The cells below load the Python-generated Morris measures and plots for the full design (with `ITILLAGE`) and for the repeat run without `ITILLAGE`. They provide a numeric comparison of `mu*` (mean absolute elementary effect) and `sigma` between the two runs so you can see how dominant `ITILLAGE` is and how rankings change when it is removed.

Short interpretation:

- `ITILLAGE` dominates the full run (very large `mu*`). Removing it re-ranks other factors; look at `delta_mu_abs` above to see which factors increase in relative influence.
- Use the no-`ITILLAGE` measures to decide which factors are worth deeper analysis once management (tillage) is held fixed.
- For submission, embed these tables/figures or export them as CSV/PNG — they are already saved in the repository root and `figs/` directory.